LAPORAN PRAKTIKUM

TEKNOLOGI IOT 2



Disusun Oleh :

Muhamad Ramadhani (25782045)

Dosen Pembimbing :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T..

Agus Ambarwari, S.Pd., M.Kom

TEKNOLOGI REKAYASA INTERNET

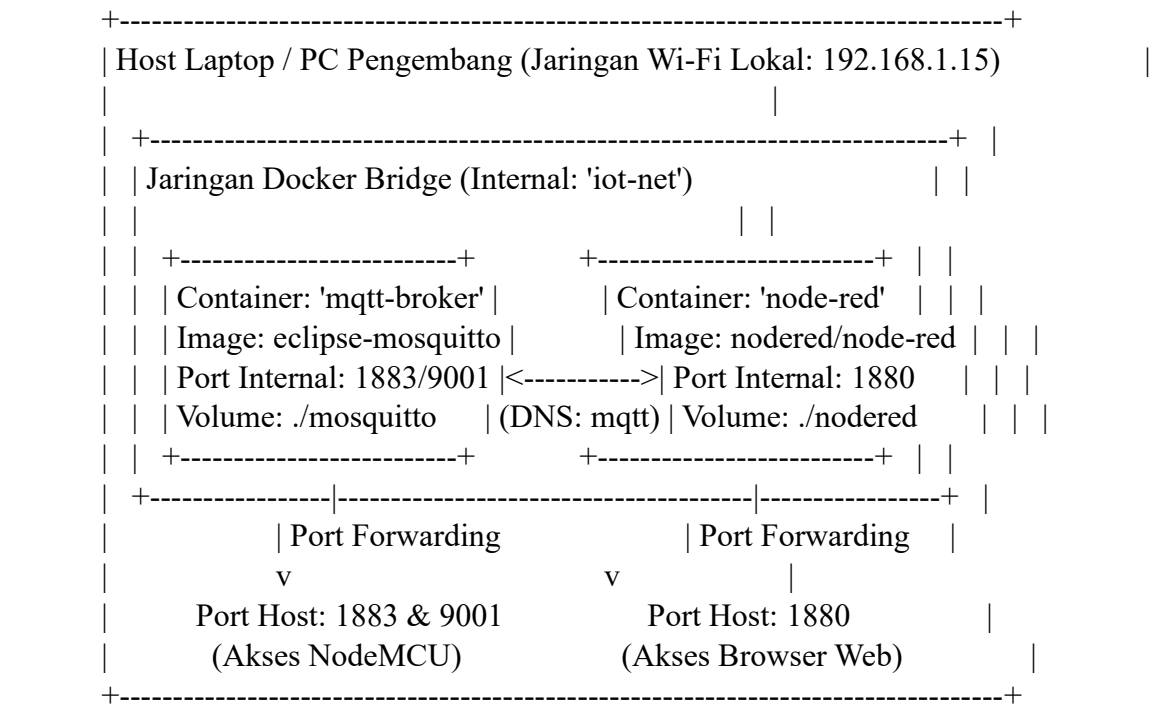
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

Pada praktikum ini, kita akan membangun infrastruktur server IoT lokal terpadu (Mosquitto MQTT Broker dan Node-RED) secara otomatis hanya menggunakan berkas deklaratif Docker Compose.



Langkah Kerja:

1. Buka File Explorer di laptop Anda, buat direktori kerja baru bernama: **iot-server** (misalnya pada direktori D:/iot-server atau C:/Users/NamaAnda/iot-server).
2. Di dalam folder iot-server, buat struktur sub-direktori berikut secara manual:
3. iot-server/
4. ├── mosquitto/
5. │ └── config/
6. Buka teks editor (VS Code atau Notepad), buat berkas baru bernama **mosquitto.conf**, simpan di dalam folder iot-server/mosquitto/config/, lalu masukkan konfigurasi parameter berikut:

```
# ==================================================================================
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ==================================================================================

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous true
```

```
# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data/

# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

4. Kembali ke folder induk iot-server, buat berkas baru bernama **docker-compose.yml**, lalu salin konfigurasi layanan terpadu berikut:

```
services:
  # ============================================================
  # 1. Layanan Broker MQTT: Eclipse Mosquitto
  # ============================================================
  mqtt:
    image: eclipse-mosquitto:latest
    container_name: mqtt-broker
    restart: unless-stopped
    ports:
      - "1883:1883"  # Port MQTT murni untuk mikrokontroler
      - "9001:9001"  # Port WebSockets untuk dashboard web
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log

  # ============================================================
  # 2. Layanan Visual Rule Engine: Node-RED
  # ============================================================
  nodered:
    image: nodered/node-red:latest
    container_name: node-red
    restart: unless-stopped
    ports:
      - "1880:1880"  # Port antarmuka web builder Node-RED
    environment:
      - TZ=Asia/Jakarta
    volumes:
      - ./nodered:/data
    depends_on:
      - mqtt
```

5. Jalankan **Terminal** (Linux/macOS) atau **PowerShell / Command Prompt** (Windows), arahkan direktori aktif ke dalam folder iot-server:

```
cd D:\iot-server
```

6. Eksekusi perintah berikut untuk mengunduh (*pull*) *image* dan menjalankan kontainer di latar belakang (*detached mode*):

docker compose up -d

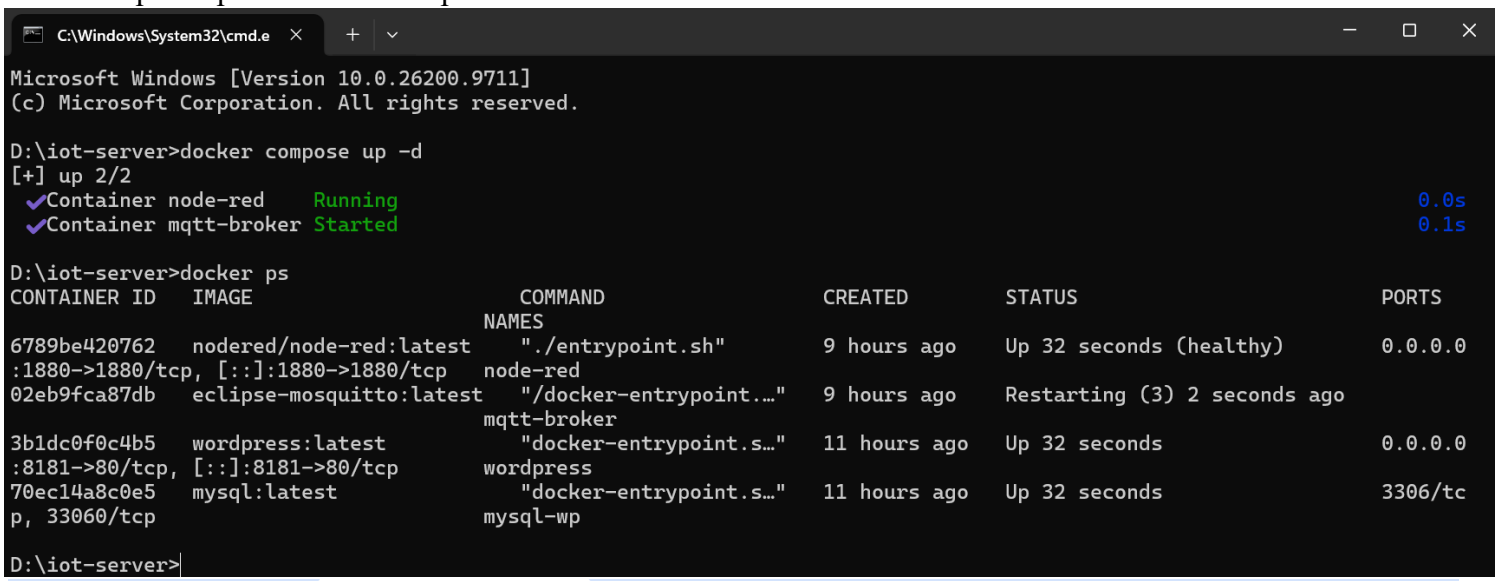
(Catatan: Pada versi Docker lama, gunakan perintah docker-compose up -d).

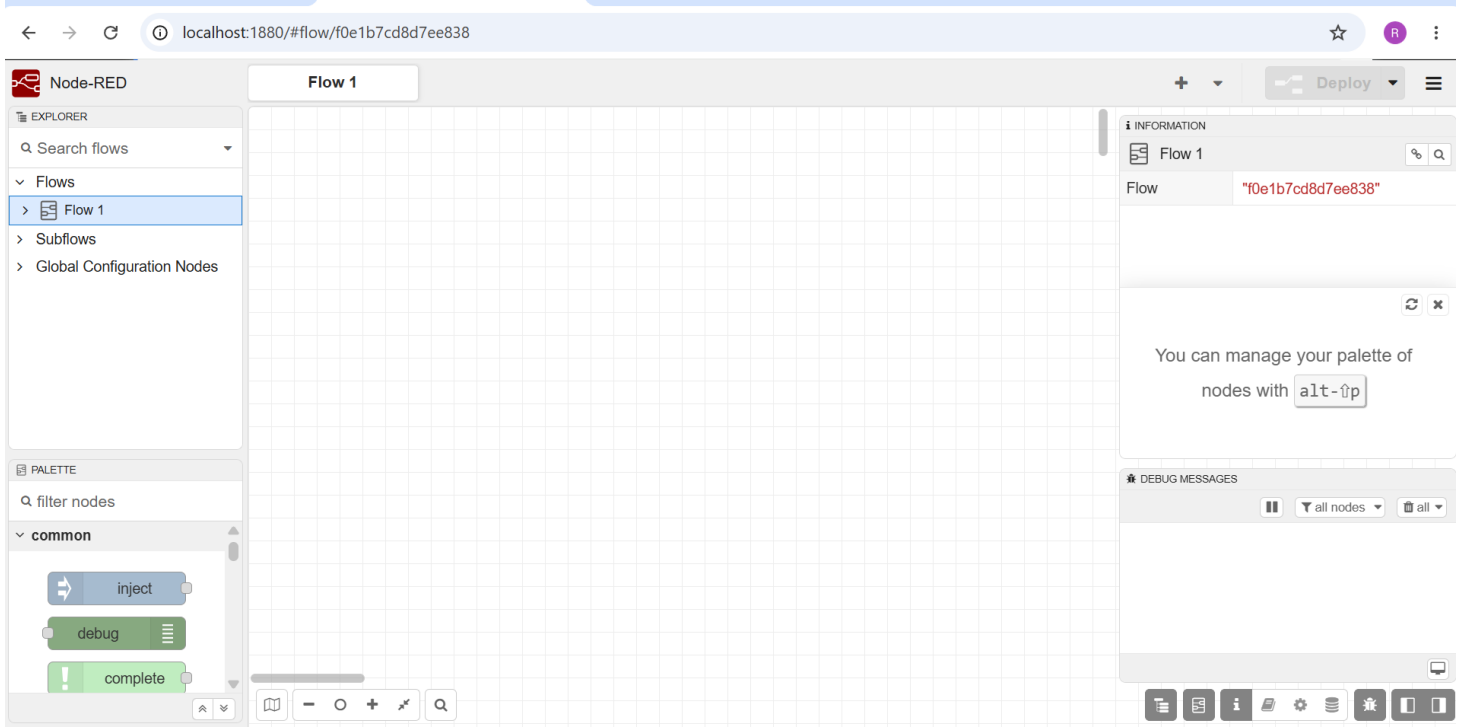
7. Tunggu hingga terminal menampilkan status [+] Running 2/2 dengan keterangan hijau Started.
8. Verifikasi bahwa kedua kontainer telah aktif dan berjalan normal dengan mengecek daftar proses kontainer:

docker ps

Pastikan kolom STATUS untuk kontainer mqtt-broker dan node-red menunjukkan Up (healthy/running).

9. Buka peramban web pada laptop Anda dan ketikkan alamat URL: **http://localhost:1880**.
   Jika antarmuka visual kanvas Node-RED berhasil tampil di layar, maka fondasi server IoT lokal Anda telah siap beroperasi secara sempurna!





7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

Setelah server lokal berjalan di laptop, langkah berikutnya adalah mengarahkan NodeMCU ESP8266 dari broker publik ke alamat IP lokal laptop Anda dalam jaringan Wi-Fi lokal.

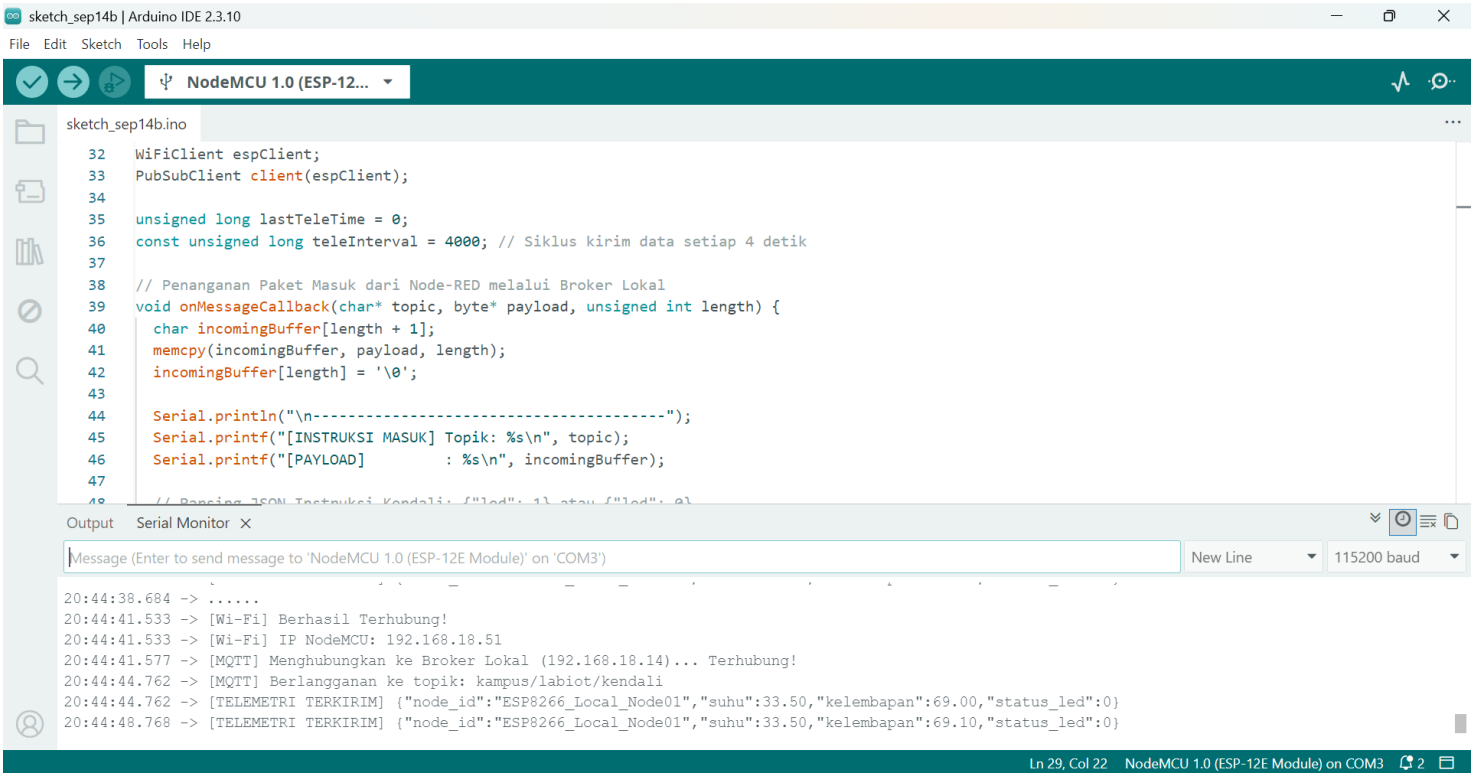
Skema Rangkaian Perangkat Keras:

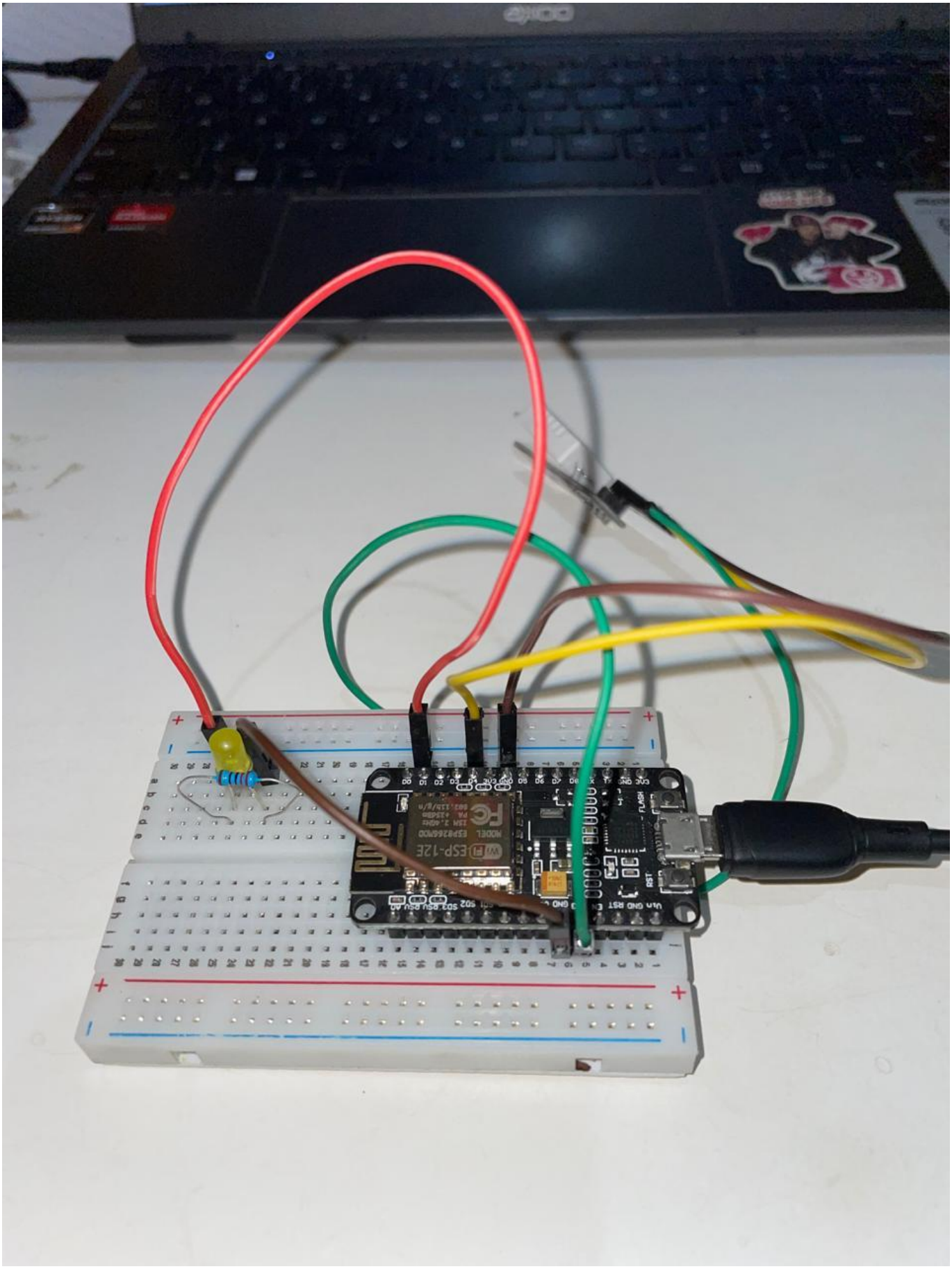
Rangkaian perangkat keras pada praktikum ini identik dengan konfigurasi pada Modul 6 (hanya menggunakan sensor DHT dan indikator LED):

- **Sensor DHT11/DHT22:**
  - Pin VCC →Pin 3V3 NodeMCU
  - Pin GND →Pin GND NodeMCU
  - Pin Data →Pin **D4 (GPIO 2)**
- **LED Indikator Aktuator:**
  - Anoda (kaki panjang) →Pin **D1 (GPIO 5)**
  - Katoda (kaki pendek) →Resistor 220 Ω →Pin GND

Langkah Pemrograman:

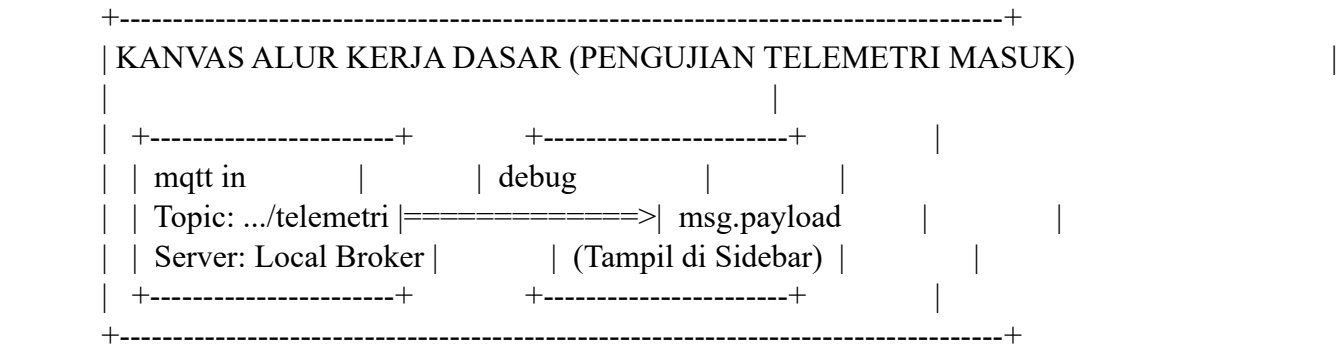
1. Cari tahu **Alamat IPv4 Lokal** laptop Anda yang terhubung ke jaringan Wi-Fi:
   - **Windows:** Buka CMD / PowerShell, ketik ipconfig. Temukan bagian *Wireless LAN adapter Wi-Fi*, lalu catat baris **IPv4 Address** (misalnya: 192.168.1.15).
   - **macOS / Linux:** Buka terminal, ketik ifconfig atau ip a (misalnya: 192.168.1.15).
2. **[PERINGATAN PENTING FIREWALL WINDOWS]:**
   Sistem keamanan *Windows Defender Firewall* sering kali secara otomatis memblokir koneksi masuk (*inbound connection*) dari port 1883. Jika NodeMCU gagal terhubung ke laptop, buka **Windows Defender Firewall with Advanced Security**, klik **Inbound Rules > New Rule**, pilih **Port > TCP**, masukkan port 1883, lalu pilih **Allow the connection**.





7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

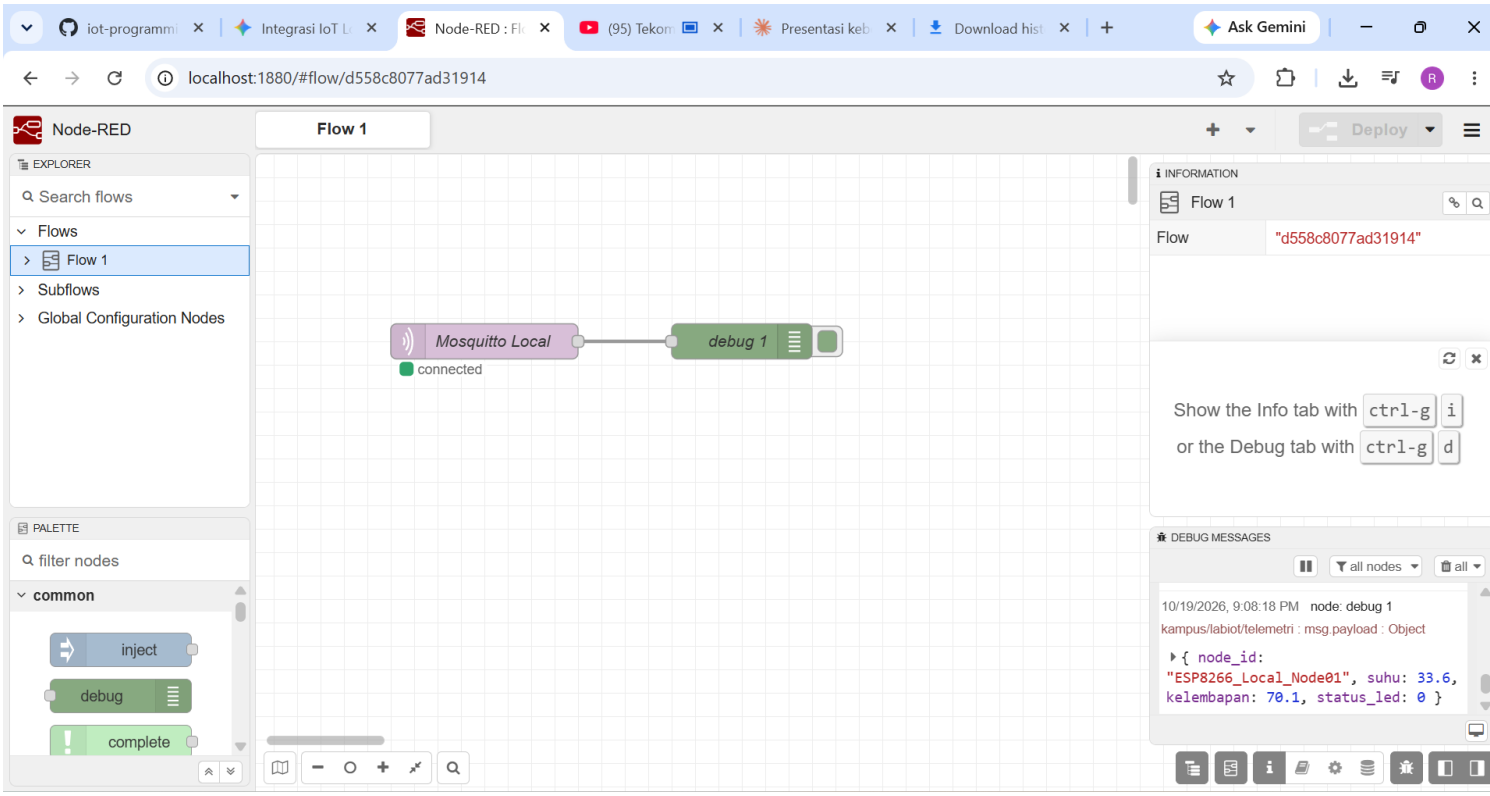
Pada praktikum ini, kita akan menangkap dan membedah aliran data telemetri yang dikirimkan oleh NodeMCU menggunakan antarmuka kanvas Node-RED.



Langkah Kerja:

1. Buka peramban web dan akses antarmuka Node-RED di **http://localhost:1880**.
2. **Menambahkan Konfigurasi Server MQTT Broker:**
   - Di sebelah kiri (*Palette*), temukan kategori **network**, lalu klik dan seret (*drag*) node **mqtt in** ke tengah kanvas kerja.
   - Klik ganda (*double-click*) pada node mqtt in tersebut untuk membuka panel pengaturannya:
     - Pada baris **Server**, klik ikon pensil ( ) untuk membuat konfigurasi broker baru.
     - **Name:** Mosquitto Local
     - **Server:** Ketik **mqtt**
       (PENTING: Jangan gunakan localhost atau IP laptop! Karena Node-RED dan Mosquitto berada dalam satu jaringan Docker Compose yang sama, kontainer Node-RED dapat memanggil broker secara langsung menggunakan nama servis deklaratifnya, yaitu mqtt).
     - **Port:** 1883
     - Klik tombol merah **Add** di pojok kanan atas dialog.
3. **Mengonfigurasi Parameter Topik:**
   - Pada baris **Topic**, masukkan: kampus/labiot/telemetri
   - Pada baris **Output**, ubah dari *auto-detect* menjadi **a parsed JSON object**. (Fitur ini secara otomatis mengurai teks string JSON menjadi objek JavaScript hierarkis tanpa perlu menulis fungsi *parser* manual).
   - Klik tombol **Done**.
4. **Menambahkan Node Pemantau (*Debug Node*):**
   - Pada *Palette* sebelah kiri di kategori **common**, seret node **debug** ke kanvas.
   - Tarik garis (*wire*) dari titik port keluaran di sisi kanan node mqtt in ke port masukan di sisi kiri node debug.
5. **Menerapkan Alur Kerja (*Deploy*):**
   - Klik tombol merah **Deploy** di sudut kanan atas antarmuka.
   - Amati bahwa di bawah node mqtt in muncul teks indikator hijau bertuliskan **connected**.
6. **Menganalisis Aliran Data:**
   - Klik tab ikon kumbang ( ) di bilah sisi kanan (*Sidebar*) untuk membuka panel **Debug Messages**.
   - Anda akan melihat objek JSON masuk setiap 4 detik secara *real-time*:
   - kampus/labiot/telemetri : msg.payload : Object
   - {
   - node_id: "ESP8266_Local_Node01",

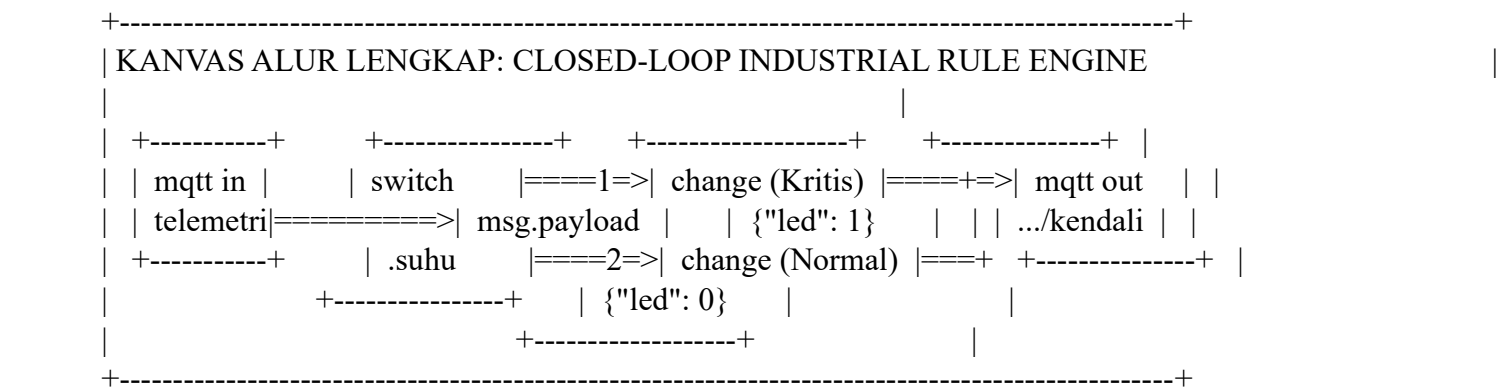
- suhu: 29.8,
- kelembapan: 62.4,
- status_led: 0
- }
- Klik tanda panah segitiga kecil di samping teks Object untuk mengeksplorasi hierarki pasangan kunci dan nilai data.



7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

Pada praktikum ini, kita akan membangun *Closed-Loop Rule Engine* otonom:

- **JIKA** nilai suhu yang dibaca oleh sensor melebihi ambang batas kritis 31.0°C: Node-RED secara otomatis memublikasikan paket JSON {"led": 1} untuk menyalakan LED indikator peringatan bahaya di pin D1.
- **JIKA** suhu berada dalam batas normal (≤ 31.0°C): Node-RED memublikasikan paket JSON {"led": 0} untuk memadamkan LED indikator.



Langkah Kerja:

1. **Membuat Node Percabangan Logika (switch):**
   - Pada kategori **function**, seret node **switch** ke kanvas.
   - Hubungkan port keluaran node mqtt in ke port masukan node switch.
   - Klik ganda node switch untuk menyetel aturan logika evaluasi:
     - **Property:** Ganti msg.payload menjadi **msg.payload.suhu**.
     - **Aturan 1 (Kondisi Panas):** Pilih operator **>**, lalu masukkan angka: **31**.
     - Klik tombol **+ add** di bagian bawah untuk menambahkan aturan kedua.
     - **Aturan 2 (Kondisi Aman):** Pilih operator **<=**, lalu masukkan angka: **31**.
     - Klik **Done**. Perhatikan bahwa node switch kini memiliki dua port keluaran bernomor 1 (atas) dan 2 (bawah).
2. **Membuat Node Transformasi Muatan Panas (change):**
   - Seret node **change** dari kategori *function* ke kanvas di sebelah kanan atas node switch.
   - Hubungkan port keluaran 1 (suhu > 31) dari node switch ke masukan node change pertama ini.
   - Klik ganda node change:
     - **Name:** Nyalakan LED Indikator
     - **Set:** msg.payload
     - **To:** Ubah tipe data dari string (az) menjadi **JSON** ({ }), lalu masukkan nilai:
     - {
     - "led": 1

}

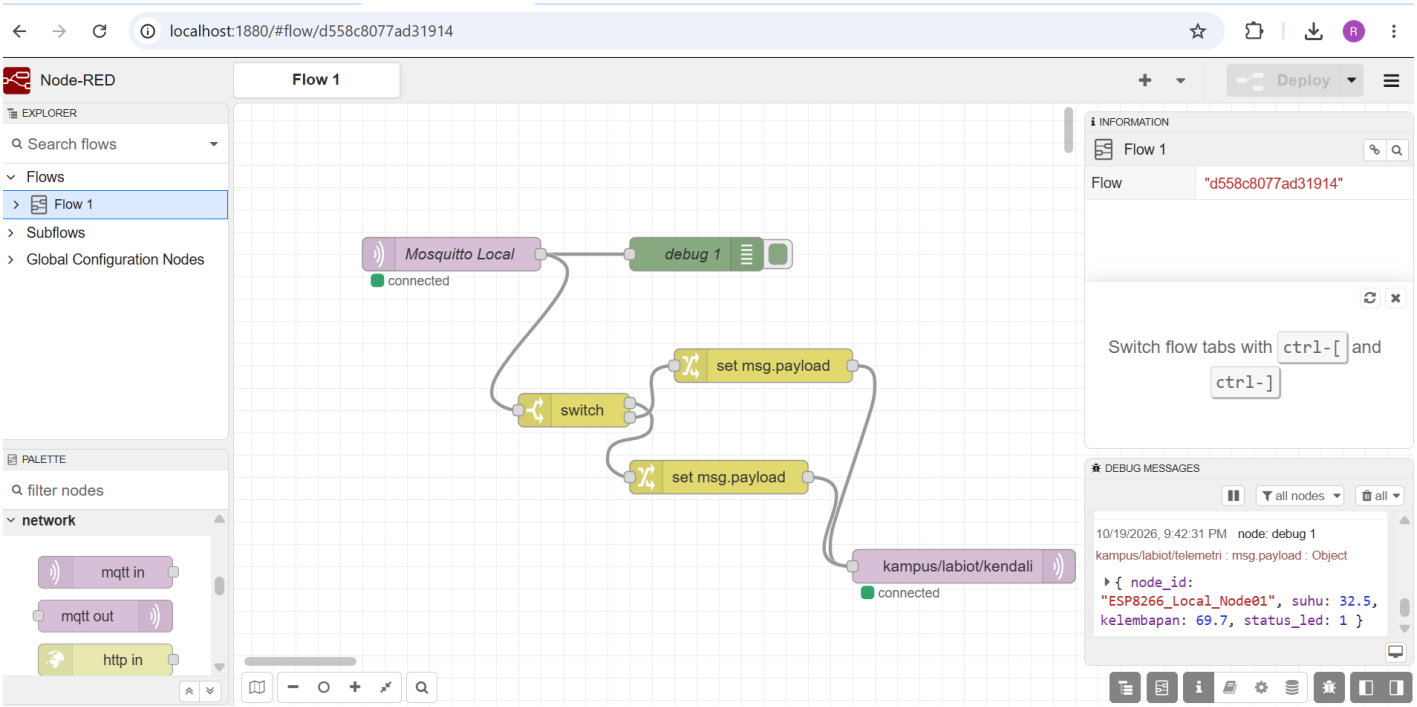
     - Klik **Done**.
3. **Membuat Node Transformasi Muatan Normal (change):**
   - Seret node **change** kedua ke kanvas di sebelah kanan bawah node switch.
   - Hubungkan port keluaran 2 (suhu ≤ 31) dari node switch ke masukan node change kedua ini.
   - Klik ganda node change:
     - **Name:** Padamkan LED Indikator
     - **Set:** msg.payload
     - **To:** Ubah tipe data menjadi **JSON** ({ }), lalu masukkan nilai:

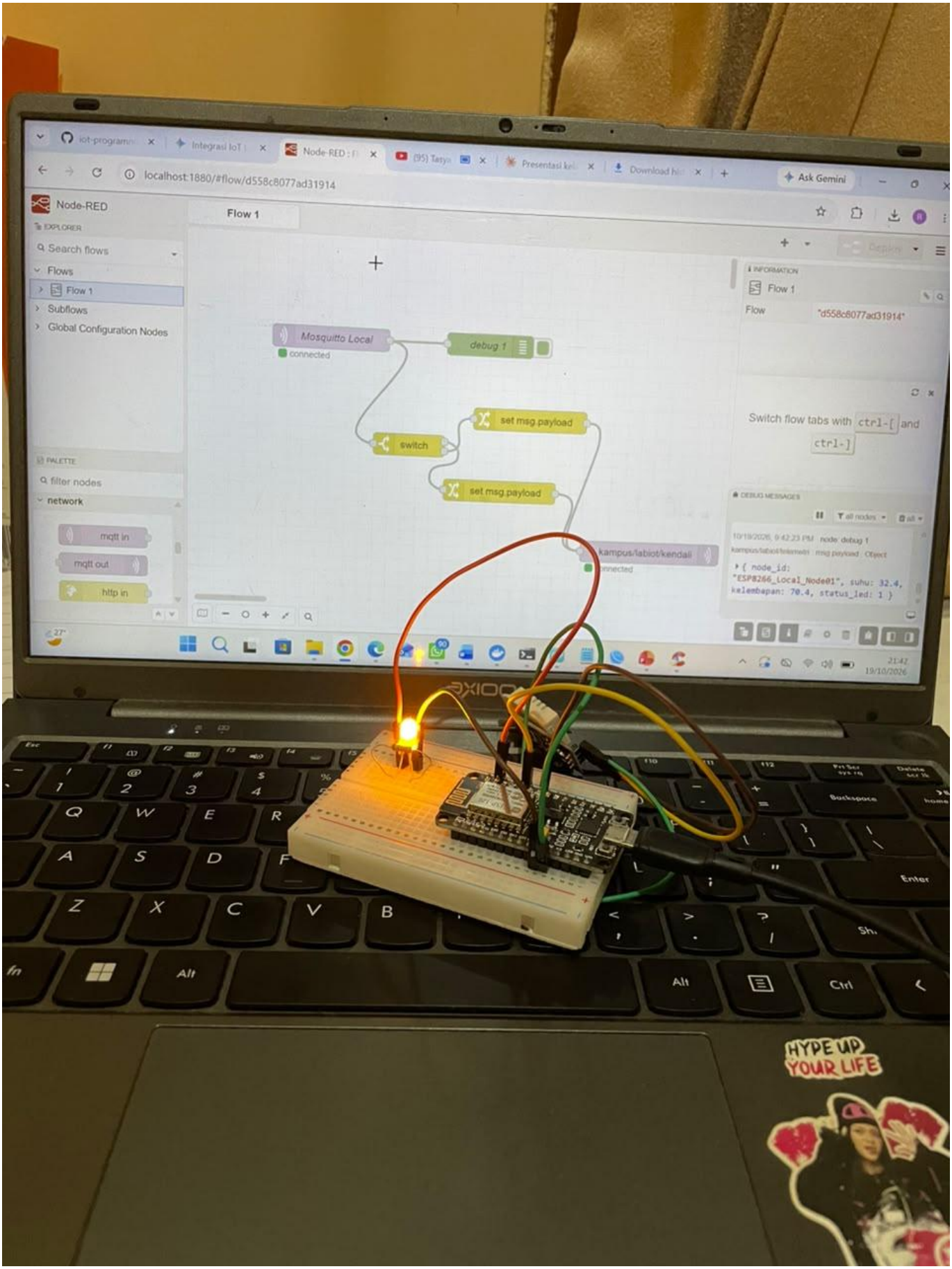
        - {
        - "led": 0

}

        - Klik **Done**.

4. **Menerbitkan Instruksi Balik ke Broker (mqtt out):**
    - Pada kategori **network**, seret node **mqtt out** ke kanvas.
    - Hubungkan keluaran dari node change pertama dan node change kedua secara bersamaan ke port masukan node mqtt out.
    - Klik ganda node mqtt out:
        - **Server:** Pilih Mosquitto Local.
        - **Topic:** kampus/labiot/kendali
        - **QoS:** 0 (atau 1)
        - **Retain:** false
        - Klik **Done**.
5. **Menerapkan Perubahan (*Deploy*):**
    - Klik tombol merah **Deploy** di kanan atas antarmuka.
6. **Pengujian Fisik Sistem Otomasi Tertutup:**
    - Amati status awal: Karena suhu ruangan umumnya berkisar antara $26°C - 29°C (\leq 31°C)$, LED indikator berada dalam kondisi padam.
    - Dekatkan sumber panas ringan (seperti ujung jari Anda yang ditempelkan kuat pada sensor DHT, atau hembusan napas hangat) hingga pembacaan suhu pada panel Debug merangkak naik melewati 31.0°C.
    - **Hasil Pengamatan:** Seketika suhu melampaui 31.0°C, Node-RED mengeksekusi port 1, mengirimkan payload {"led": 1}, dan LED indikator bahaya di pin D1 seketika menyala terang!
    - Lepaskan jari Anda dan biarkan suhu mendingin kembali ke bawah 31.0°C. LED indikator akan padam kembali secara otomatis.




Node-RED
Flow 1
Search flows
Flows
Subflows
Global Configuration Nodes
Mosquitto Local
connected
debug 1
switch
set msg.payload
set msg.payload
kampus/labiot/kendali
connected
Switch flow tabs with ctrl-[and ctrl-]
mqtt in
mqtt out
http in
HYPE UP YOUR LIFE

Latihan 1: Analisis Jaringan Internal Docker & Service Discovery

1. **Penyebab Kegagalan Koneksi localhost:** Atribut localhost (127.0.0.1) di dalam kontainer merujuk pada antarmuka *loopback* lokal milik kontainer node-red itu sendiri, bukan komputer host atau kontainer broker. Karena servis broker Mosquitto berjalan secara terisolasi di dalam kontainer lain (mqtt-broker), kontainer Node-RED menolak koneksi (*Connection Refused*) sebab tidak ada servis MQTT yang aktif di port 1883 pada kontainernya sendiri.
2. **Mekanisme Docker Embedded DNS & Bridge Network:** Saat kontainer dijalankan dalam *User-Defined Bridge Network* via Docker Compose, Docker otomatis mengaktifkan fitur *Embedded DNS Server* internal (di alamat IP 127.0.0.11). DNS server ini mencatat *service name* yang didefinisikan pada berkas docker-compose.yml (seperti mqtt). Ketika Node-RED memanggil alamat mqtt, Docker Embedded DNS mentranslasikan nama tersebut secara otomatis menjadi alamat IP internal kontainer Mosquitto (*service discovery*).

Latihan 2: Penanganan Masalah Konektivitas Jaringan & Subnet Firewall

1. **Penyebab NodeMCU Gagal Terhubung:** NodeMCU (192.168.43.102 / subnet /24) dan Laptop (192.168.1.15 / subnet /24) berada pada dua segmen jaringan (subnet) terpisah. Tanpa adanya perangkat *router* yang melakukan rute antarsubnet (*inter-VLAN / inter-subnet routing*), paket data broadcast dan komunikasi TCP/IP dari NodeMCU tidak dapat melewati batas jaringan menuju IP laptop.
2. **Langkah Teknis Penyelarasan Subnet:**
    - Sambungkan NodeMCU dan Laptop ke titik akses Wi-Fi (*Access Point*) atau *hotspot* yang sama.
    - Pastikan kedua perangkat memperoleh alokasi IP dinamis (DHCP) pada subnet yang seragam (misalnya sama-sama berada di segmen 192.168.1.x atau 192.168.43.x).
    - Perbarui variabel mqtt_server pada kode program Arduino IDE menggunakan alamat IP baru laptop yang berada di subnet tersebut.

Latihan 3: Implementasi Filter Histeresis pada Rule Engine

Berikut kode JavaScript untuk **Function Node** di Node-RED guna mencegah fluktuasi (*flickering*) status LED menggunakan batas histeresis:

JavaScript

```
// Membaca status LED sebelumnya dari memori kontekstual (default: 0 / OFF)
let state = context.get('state') || 0;
let suhu = msg.payload.suhu;

// Logika histeresis dua ambang batas
if (suhu > 31.5) {
   state = 1; // Menyala hanya jika suhu menembus > 31.5°C
} else if (suhu < 30.0) {
   state = 0; // Padam hanya jika suhu turun di bawah < 30.0°C
}

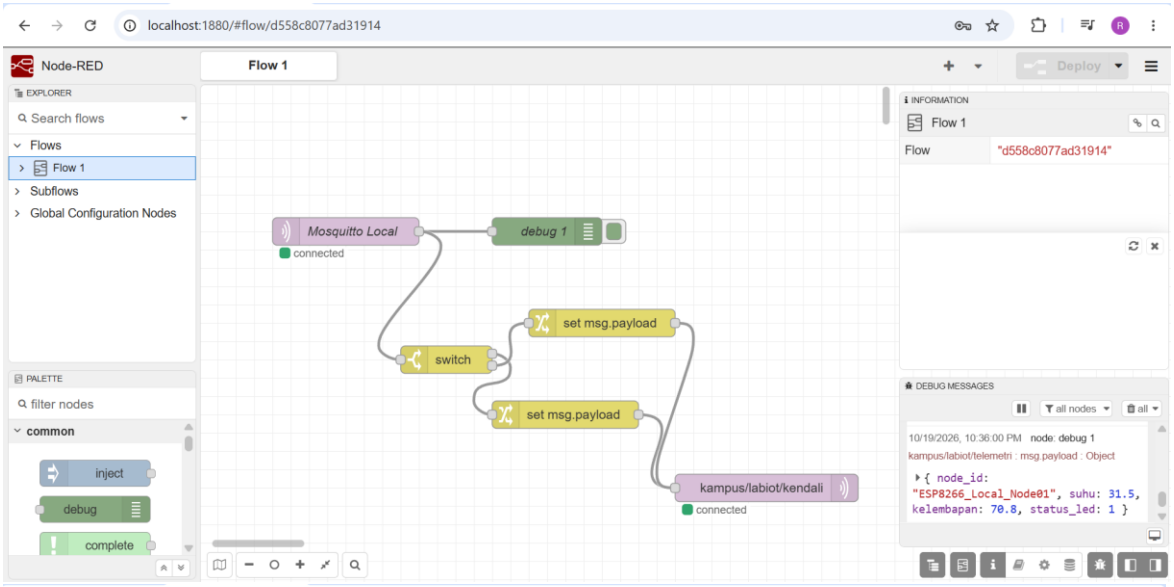
// Menyimpan status terbaru ke memori kontekstual
context.set('state', state);

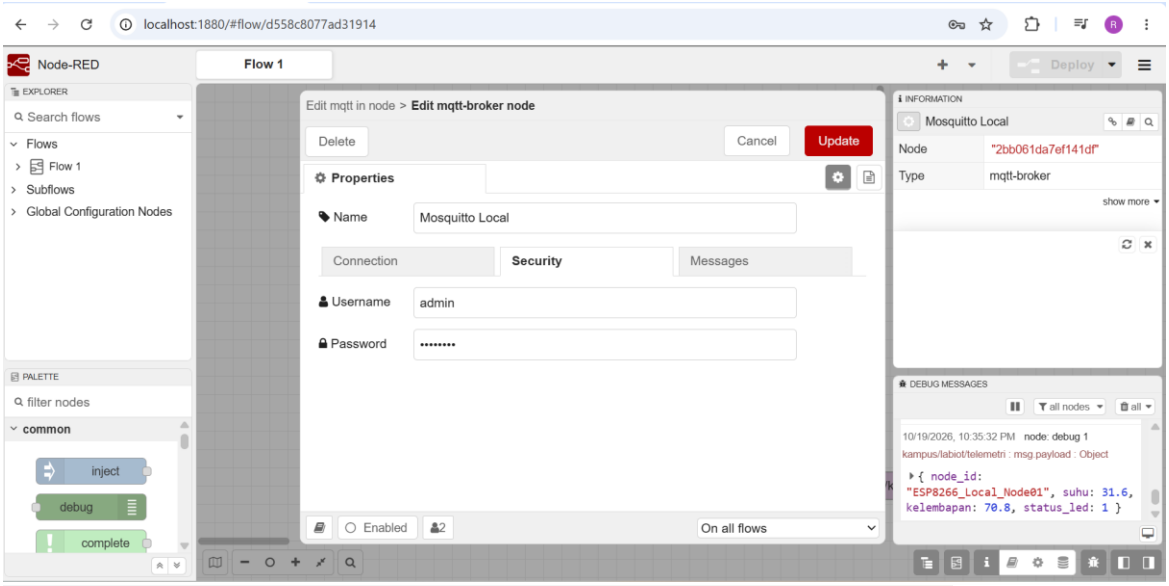
// Mengirimkan objek payload baru
msg.payload = { "led": state };
return msg;
```

Latihan 4: Analisis Mekanisme Volume Mount & Persistensi Data

1. **Dampak Tanpa Volume Mount:** Sifat dasar kontainer Docker adalah *ephemeral* (sementara). Jika baris volumes dihapus dan kontainer dihentikan menggunakan docker compose down, seluruh alur (*flow*) visual, node yang dikonfigurasi, dan kredensial di Node-RED akan terhapus permanen dari memori sistem karena data hanya tersimpan di *writable layer* sementara kontainer.
2. **Perbedaan Bind Mount vs Named Volume:**
    - **Bind Mount (./nodered:/data):** Memetakan direktori atau berkas fisik spesifik dari komputer host langsung ke dalam direktori kontainer. Jalur direktori ditentukan secara eksplisit oleh pengguna dan berkasnya dapat diedit langsung melalui File Explorer host.
    - **Named Volume (volume_name:/data):** Dikelola sepenuhnya oleh internal Docker di dalam area penyimpanan khusus sistem Docker (/var/lib/docker/volumes/). Isinya diabstraksi dari akses langsung pengguna dan dikelola menggunakan perintah

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)





File Edit View

```
# ------------------------------------------------------------------------------
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ------------------------------------------------------------------------------

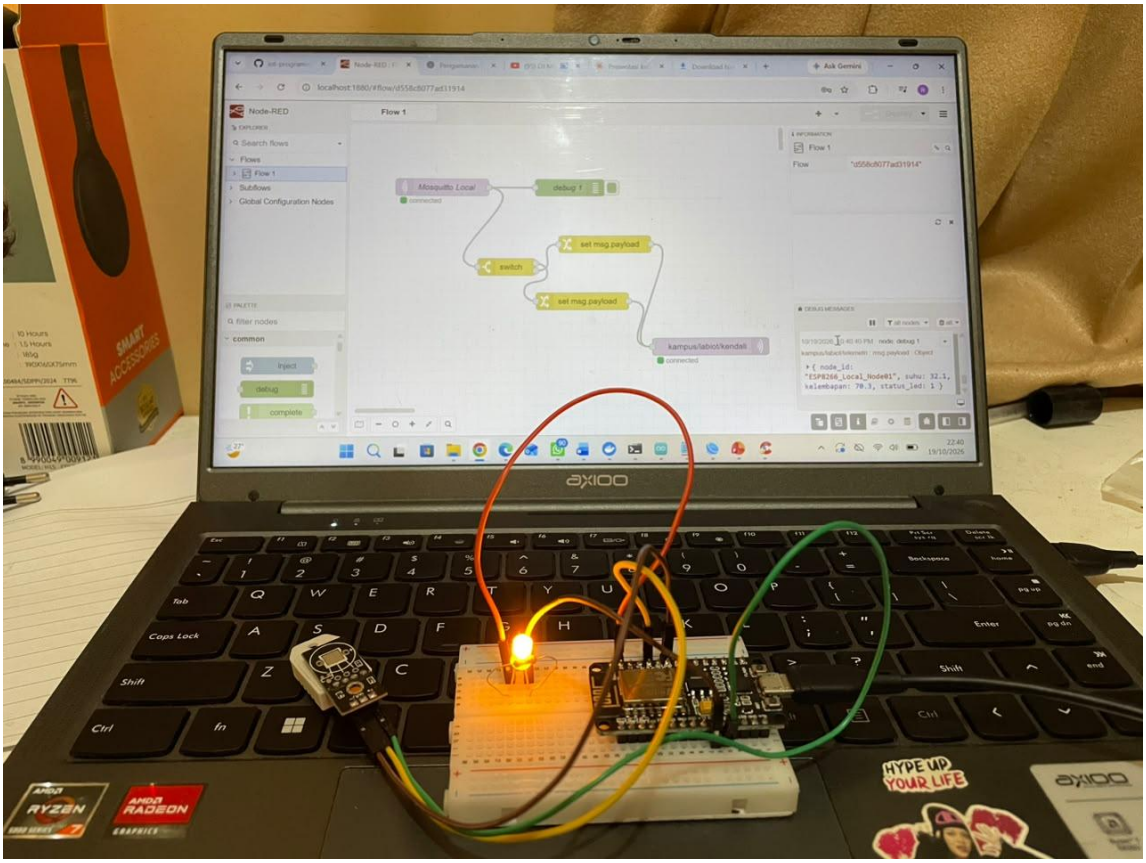
# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous false
password_file /mosquitto/config/password.txt

# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data

# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```



Pada praktikum 7.7, pengamanan server IoT dilakukan dengan menerapkan autentikasi pada MQTT Broker Mosquitto menggunakan username dan password. Konfigurasi allow_anonymous false digunakan untuk menolak koneksi tanpa autentikasi, sedangkan password_file /mosquitto/config/password.txt digunakan untuk menentukan lokasi file kredensial yang menyimpan hash password pengguna. Pada proses konfigurasi, sempat terjadi kegagalan karena Mosquitto tidak dapat membaca file password akibat masalah izin akses. Masalah tersebut berhasil diatasi dengan menyesuaikan izin file sehingga broker dapat berjalan normal. Selanjutnya, Node-RED dikonfigurasi menggunakan username admin dan password yang sesuai agar dapat terhubung kembali ke broker. Program ESP8266 juga perlu menggunakan kredensial yang sama melalui fungsi client.connect() agar mikrokontroler dapat melakukan koneksi MQTT, mengirim data sensor, menerima perintah, dan mengendalikan LED. Dengan penerapan autentikasi ini, akses anonim dapat dicegah sehingga komunikasi antara ESP8266, Mosquitto, dan Node-RED menjadi lebih aman.